

Superscalar Architecture

Week 3: Register Renaming, the ROB, and First Verilog

May 27, 2026

Contents

Week 3 in 60 Seconds	4
1 Week 3 — Register Renaming, ROB, First Verilog	6
2 Comprehensive Theory 1: From Tags to Physical Registers [~25 min]	7
2.1 What We Built in Week 2	7
2.2 The Problem at Scale	7
2.3 The Reframe: Physical Register File	7
2.4 Why This Is the Same Algorithm as Tomasulo	8
2.5 A Side-by-Side Example	9
3 Comprehensive Theory 2: The Reorder Buffer (ROB) [~25 min]	11
3.1 Structure	11
3.2 The Three Stages of an Instruction in the ROB	12
3.3 How the ROB Provides Precise Exceptions	12
3.4 How the ROB Provides Branch Recovery	13
3.5 Why Track “Old Physical Destination”	13
4 Comprehensive Theory 3: The Rename Map and Free List [~20 min]	14
4.1 The Rename Map	14
4.2 The Free List	14
4.3 Why Free List Discipline Matters	15
4.4 A Cycle-Level View	15
5 Comprehensive Theory 4: Worked Example — Renaming in Action [~25 min]	17
5.1 Setup	17
5.2 Cycle 1: Dispatch I1	17
5.3 Cycle 2: Dispatch I2	17
5.4 Cycle 3: Attempt to Dispatch I3 — Stall!	18
5.5 Cycle 4: I1 Completes; Cycle 5: I1 Commits	18
5.6 Cycle 5: Dispatch I3 (finally)	18
5.7 Key Takeaways	19
6 Verilog Design Philosophy — Before You Write Any Code [~15 min]	20
6.1 We Are Using Verilog-2001, Not SystemVerilog	20
6.2 Hardware Is Not Software	20
6.3 Synthesizable vs. Non-synthesizable Verilog	21
6.4 Naming Conventions for Your Sanity	21
6.5 Blocking vs. Non-blocking Assignment	21
6.6 Reset Strategy	22

7	Module 1: <code>register_file.v</code> — The Physical Register File [~30 min]	23
7.1	Specification	23
7.2	Port List	24
7.3	Why Each Port Looks the Way It Does	24
7.4	Implementation	25
7.5	What Could Go Wrong	27
8	Module 2: <code>rename_unit.v</code> — Rename Map + Free List [~35 min]	29
8.1	Specification	29
8.2	Port List	29
8.3	Why Each Port Looks the Way It Does	30
8.4	Internal Structure	30
8.5	Implementation	30
8.6	The Subtle Bits	32
8.7	What Could Go Wrong	32
9	Testbenches — Verifying Your Modules [~25 min]	34
9.1	What Makes a Good Testbench	34
9.2	Testbench Skeleton (Register File)	34
9.3	Required Test Cases for the Register File	37
9.4	Required Test Cases for the Rename Unit	38
9.5	Running the Testbench in ModelSim	38
10	Week 3 Deliverables — What to Submit	39
A	Appendix: Solutions to Concept Checks	41

Week 3 in 60 Seconds

A one-page summary. Theory is lighter this week; coding begins.

The bridge from Week 2: Classical Tomasulo uses reservation-station names as tags. **Modern processors use physical-register-file indices as tags.** These are conceptually identical (both are unique identifiers for an in-flight value), but the modern approach scales to hundreds of in-flight instructions and integrates cleanly with the Reorder Buffer.

The new mental model:

- There are now **two register files**: the *architectural* one (32 entries, what the programmer sees, exists only conceptually) and the *physical* one (48–64 entries, the actual hardware storage).
- The **Rename Map** is a 32-entry table that says: “architectural register R is currently stored in physical register P.”
- The **Free List** is a queue of physical registers not currently mapped to any architectural register.
- When an instruction writes to architectural register R: pop a physical register P from the free list, update the rename map ($R \rightarrow P$), use P as the destination tag.
- When an instruction reads from architectural register R: look up the rename map, find P, the value lives in physical register file slot P.

The Reorder Buffer (ROB) — conceptual only this week:

- A circular buffer of entries, allocated in program order at dispatch.
- Each entry tracks: instruction, destination architectural reg, destination physical reg, old physical reg, ready bit, exception bit.
- Instructions complete (write to physical RF) in execution order; they **commit** (become architecturally visible) at the ROB head, in program order.
- On a branch mispredict or exception: flush all entries after the offender; restore rename map.
- **We will build the ROB in Verilog in Week 6, not this week.** It needs intra-bundle bypass logic (Week 5) and CDB arbitration logic (Week 6) that we haven’t introduced yet. The ROB is taught conceptually now because it explains *why* the rename unit you build this week tracks an “old physical destination” field for every dispatched instruction — the ROB will need it for rollback.

Three reasons the ROB is necessary (and Tomasulo lacked):

1. **Precise exceptions:** architectural state at the moment of a fault is well-defined.

2. **Branch recovery:** speculative work can be undone.
3. **In-order commit:** debuggable, predictable architectural behavior.

This week's Verilog deliverables: Two modules in Verilog-2001 (.v files): `register_file.v` (4 read ports + 2 write ports, with per-register ready bits and write-before-read bypass) and `rename_unit.v` (rename map + free list). Plus testbenches for both. Section 5 explains the design philosophy; Sections 6–8 walk through each module and its testbench.

Suggested reading order: Sections 1–3 are theory (the reframe, the ROB, the rename map and free list). Section 4 ties them together with a worked example. Sections 5–7 are the Verilog: design philosophy, then the register file, then the rename unit. Section 8 is the testbench. Section 9 is the deliverable.

1 Week 3 — Register Renaming, ROB, First Verilog

Goal

Translate the Tomasulo insight from Week 2 into the modern implementation style used by every real processor today: explicit register renaming with a physical register file, plus a reorder buffer for in-order commit (conceptual this week). Then write your first Verilog modules in this project: a parameterizable 4R2W register file and a rename unit.

Mentee Work (9–10 hours)

- Read Sections 1–4 (theory, ~3 hours). Concept Checks as you go.
- Watch Onur Mutlu’s lectures on register renaming and ROB (ETH Computer Architecture series, on YouTube). ~1.5 hours.
- Read Sections 5–8 (Verilog design + module specifications, ~1 hour).
- **Code, simulate, and verify:** `register_file.v` and `rename_unit.v` with both testbenches. ~4–5 hours. This is your first real coding effort in the project.

Deliverable

- In your repo’s `rtl/` folder: `register_file.v`, `rename_unit.v`.
- In your repo’s `tb/` folder: `tb_register_file.v`, `tb_rename_unit.v`.
- In your repo’s `sim/` folder: ModelSim waveform screenshots (PNG) showing key test scenarios working.
- Updated `learning_log.md` with what you learned, what broke, what you fixed.

What We Will Look For

- Correct port widths and counts (4 read, 2 write on the register file).
- Free-list management that never gives the same physical register to two architectural registers at once.
- Testbench that covers the tricky case: back-to-back writes to the same architectural register must allocate different physical registers.
- Code is in `rtl/` (synthesizable only, no `$display`) and testbenches in `tb/` (where `$display/$readmemh` are fine).
- Pure Verilog-2001(normal Verilog) only (no `logic`, no `always_ff`, no `always_comb`). The Quartus + ModelSim-Intel Starter toolchain is most reliable in plain Verilog.

2 Comprehensive Theory 1: From Tags to Physical Registers [~25 min]

This is the most important conceptual section of Week 3. Everything else builds on it.

2.1 What We Built in Week 2

In classical Tomasulo, the question “where is the current value of `$f0`?” was answered by the Register Status Table:

- Either $RST[F0] = \text{“no in-flight producer,”}$ and the value lives in the architectural register file slot `F0`; or
- $RST[F0] = \text{“RS\#7,”}$ meaning reservation station `#7` is currently computing it.

The “tag” was the reservation station name. When `RS#7` finished and broadcast on the CDB, every consumer that captured tag `RS#7` received the value.

2.2 The Problem at Scale

This works beautifully on paper. It works fine in a small machine with 6–10 reservation stations. But modern processors track **hundreds** of in-flight instructions. Intel’s recent cores have over 300 instructions in flight at any time.

Classical Tomasulo’s design has two scaling problems:

1. **Tag space = RS count.** If you have 10 RSs, you have 10 possible tags. To have 300 in-flight values, you’d need 300 RSs. But each RS needs its own tag-comparison hardware against the CDB — you can’t have 300 RS comparators all working in parallel without a huge area cost.
2. **RS-as-value-storage is wasteful.** When an RS captures an operand value, the RS itself stores the value for the duration of execution. That’s the RS doing double duty as a queue *and* as storage. Decoupling these jobs lets each be sized independently.

2.3 The Reframe: Physical Register File

The modern approach (used in BOOM, Intel Skylake, AMD Zen, Apple M-series — everyone) separates these jobs:

1. **The Physical Register File (PRF)** is a dedicated array of, say, 48 registers. It holds the actual values of in-flight computations. Each entry has a value field and a **ready bit** indicating whether the value has been computed.
2. **Reservation stations** hold instructions waiting to execute, but they store *indices* into the PRF, not values.

3. **The Rename Map** (also called the Register Alias Table, RAT) maps each architectural register to its current physical home.
4. **The Free List** is a queue of physical registers not currently mapped.

When an instruction is dispatched:

- For each *source* architectural register, look up the rename map to find which physical register holds its current value. The RS stores this physical register index.
- For the *destination* architectural register, pop a fresh physical register from the free list, and update the rename map to point to it. The RS stores this as its destination physical index. The newly-allocated physical register's ready bit is cleared (its value hasn't been computed yet).

When an instruction's operands become ready (the PRF ready bits for both source indices are set), the RS sends the instruction to its functional unit:

- The FU reads source values from the PRF using the source indices.
- Computes.
- Writes the result back to the PRF at the destination index, and sets the destination's ready bit.

Notice what the “tag” has become: a physical register file index, not an RS name. When an instruction completes, it doesn't need to “broadcast” a value the way classical Tomasulo did — because all waiters were already looking at the PRF index. Setting the PRF ready bit is sufficient; consumers can read the value directly from the PRF when they fire.

(In Week 6, we'll see that for performance reasons, a real machine still has a CDB-like broadcast that wakes up waiters in parallel rather than every RS polling every PRF ready bit. But conceptually, the PRF *is* the source of truth for values, not the RSs.)

2.4 Why This Is the Same Algorithm as Tomasulo

Read this paragraph slowly. It is the essential equivalence:

Classical Tomasulo tags = “which RS is producing this value.”

Modern renaming tags = “which physical register slot will hold this value.”

Both are unique identifiers for an in-flight result. In Tomasulo, the identifier is the producer's slot; in modern renaming, the identifier is the destination's storage slot. The dependency tracking is the same. The WAW/WAR elimination is the same.

The only differences are:

- *Where the value lives during the wait:* in Tomasulo, inside the producing RS; in modern renaming, in the PRF.

- *How many in-flight values you can support:* in Tomasulo, limited by RS count; in modern renaming, limited by PRF size, which can be much larger.

If Week 2 made sense to you, Week 3 is just a re-implementation strategy. The mental model from Week 2 still applies; the hardware just looks different.

2.5 A Side-by-Side Example

Consider `add $s0, $s1, $s2` where `$s1` and `$s2` are both ready.

Classical Tomasulo:

- Allocate RS#3 for this add.
- Look up `$s1` in RST: not in-flight, read value from arch RF, store in RS#3.V_j.
- Look up `$s2` in RST: not in-flight, read value from arch RF, store in RS#3.V_k.
- Update RST[\$s0] = RS#3.
- Eventually, ALU computes; RS#3 broadcasts (RS#3, result) on CDB.

Modern renaming:

- Pop a physical register P#17 from the free list. Clear PRF[17].ready.
- Allocate some RS slot (call it RS#3) for this add.
- Look up `$s1` in rename map: maps to P#5. Store P#5 as source-1 in RS#3.
- Look up `$s2` in rename map: maps to P#9. Store P#9 as source-2 in RS#3.
- Update rename map: `$s0` → P#17.
- When PRF[5].ready and PRF[9].ready are both set, RS#3 fires. ALU reads PRF[5] and PRF[9].
- ALU computes; result is written to PRF[17]; PRF[17].ready is set.
- Any RS waiting on P#17 now sees its operand is ready.

The shapes are identical. The labels changed. That's it.

Concept Check 1

1. Why is having more physical registers than architectural registers (e.g., 48 physical vs 32 architectural) the key enabler of high in-flight instruction counts?
2. In the modern reframe, what determines the maximum number of in-flight *writes* (not reads) to architectural registers? Why?
3. Suppose an instruction's destination is \$s0, and the rename map currently says $\$s0 \rightarrow P\#5$. We pop P#17 from the free list and update the map. What happens to P#5? Does it get freed immediately, or later? Why?

Going deeper: Hennessy & Patterson §3.6 covers explicit register renaming. Shen & Lipasti Chapter 5 has the cleanest treatment of the rename map and free list interaction.

3 Comprehensive Theory 2: The Reorder Buffer (ROB)

[~25 min]

Heads up: This section is conceptual. You will not implement the ROB in Verilog this week — it requires intra-bundle bypass logic (Week 5) and CDB arbitration (Week 6) that we have not yet introduced. The ROB module will be your Week 6 deliverable. But you must understand it now, because the rename unit you build this week stores fields (specifically, `old_phys_dest`) that exist *only* so the ROB can use them for rollback. Without understanding why, you won't understand why your rename unit's port list looks the way it does.

3.1 Structure

The ROB is a **circular buffer** of entries. Modern processors have 100–400 ROB entries. For PRAVAH, we'll use 8–16 entries (small enough to debug; large enough to be interesting).

Each ROB entry stores:

- **Valid bit:** is this entry holding a live instruction?
- **Ready bit:** has the instruction completed execution?
- **Architectural destination register** (e.g., \$s0): used at commit to update the architectural state.
- **Physical destination register** (e.g., P#17): used to find the result.
- **Old physical destination register** (e.g., P#5): the previous mapping of the arch destination, needed for rollback (more on this below).
- **Instruction type and PC:** for debugging and exception handling.
- **Exception bit:** did this instruction fault?

Two pointers manage the buffer:

- **Tail pointer:** where the next dispatched instruction will be allocated. Advances on dispatch.
- **Head pointer:** where the next instruction will commit. Advances on commit.

The ROB is **full** when $(\text{tail} + 1) \bmod N = \text{head}$. It is **empty** when $\text{head} = \text{tail}$ and no valid entries exist.

3.2 The Three Stages of an Instruction in the ROB

Stage 1: Dispatch (allocation). When the front-end has decoded an instruction and renamed it, it allocates a ROB entry at the tail pointer. Valid bit set. Ready bit cleared (instruction hasn't executed yet). The architectural dest, physical dest, and old physical dest are stored. Tail advances.

Stage 2: Completion (execution finishes). The instruction eventually finishes executing somewhere in the back-end. The functional unit writes the result to the physical register file and sets PRF[phys_dest].ready, then signals the ROB to mark this instruction's entry as "ready." *Crucially, the architectural state is not yet updated.*

Stage 3: Commit (in-order graduation). Every cycle, the ROB looks at the head entry. If it is valid and ready, the instruction commits:

- The architectural register mapping is now "official" (the rename map's mapping of arch dest to phys dest is permanent).
- The *old* physical destination (the one being displaced) is pushed back onto the free list. It is no longer needed.
- The ROB entry's valid bit is cleared. Head advances.

The key insight: **execution is out of order; commit is strictly in order.** You can have 10 instructions executing in parallel, finishing in any order, sitting in the ROB with ready bits set, all waiting patiently for the head to reach them. The ROB enforces program-order commit without serializing execution.

3.3 How the ROB Provides Precise Exceptions

Suppose instruction I_a takes an exception (e.g., divide by zero) and instructions I_{a+1} , I_{a+2} have already completed.

- I_{a+1} and I_{a+2} have their results written to the PRF and their ROB entries marked ready — but they have not committed.
- When the ROB head reaches I_a , it sees the exception bit set.
- The ROB **flushes** all entries from I_a onward (clears valid bits, resets tail to I_a).
- The rename map is rolled back: for every flushed entry, restore the arch-to-old-phys mapping using the *old physical destination* field. The free list is updated to reclaim the physical registers that were allocated to the flushed instructions.
- Architectural state is now *exactly as if only I_1 through I_{a-1} executed*. The OS exception handler runs cleanly.

3.4 How the ROB Provides Branch Recovery

Same mechanism, slightly different trigger. When a branch instruction reaches the EX stage and discovers it was mispredicted:

- Flush all ROB entries younger than the branch.
- Restore the rename map from the old-physical-destination fields of the flushed entries.
- Restart fetch from the correct branch target.

3.5 Why Track “Old Physical Destination”

This field exists *only* for recovery. Imagine an instruction at ROB entry 5 writes to \$s0, with old mapping $\$s0 \rightarrow P\#3$, new mapping $\$s0 \rightarrow P\#17$. If we flush entry 5, we must restore the rename map: $\$s0 \rightarrow P\#3$. Without storing P#3 (the old mapping) at dispatch time, we couldn’t perform the rollback. P#17 is also returned to the free list during rollback.

This is why your rename unit, this week, outputs `disp_rd_old_phys_o`. The ROB you’ll build in Week 6 will store that value in each entry. The fact that we don’t yet have a ROB module doesn’t change the fact that the rename unit must already produce the right output.

Concept Check 2

1. If the ROB has 16 entries and the average instruction takes 3 cycles to execute, what’s the maximum effective IPC you could sustain? (Hint: Little’s Law.)
2. Why does committing in program order solve the precise-exception problem, even though execution is out of order?
3. Suppose an instruction commits at ROB head, with arch dest \$s0, new phys P#17, old phys P#3. What two updates happen at commit? (Be precise about what gets written and what gets freed.)

Going deeper: Smith & Pleszkun’s 1985 paper, “Implementation of Precise Interrupts in Pipelined Processors,” invented the ROB. Short, readable, and one of the most cited papers in computer architecture.

4 Comprehensive Theory 3: The Rename Map and Free List [~20 min]

These two structures sit at the heart of the dispatch stage. Get them right, and the rest of the renamer falls into place. This week you will build them in Verilog.

4.1 The Rename Map

The rename map is a 32-entry table (one entry per architectural register), where each entry holds:

- A physical register index (e.g., $\$s0 \rightarrow P\#17$).
- A “valid” bit (in some designs, all entries are always valid because every arch register always has *some* physical home — this is the convention we’ll use).

Read port behavior. Every dispatched instruction reads up to two source mappings, plus the current mapping of its destination (needed to capture `old_phys_dest`). For a single-issue rename (this week), that’s 3 read ports on the rename map. For a 2-wide superscalar (Week 5), we’ll need 6 read ports. Reads are combinational — the result is needed in the same cycle as dispatch.

Write port behavior. Every dispatched instruction writes one destination mapping. For single-issue, that’s 1 write port. Writes are clocked on the rising edge.

The intra-cycle dependency problem (Week 5 preview). If two instructions dispatch in the same cycle and the second reads from a register the first writes, the rename map read for instruction-2 must see instruction-1’s update. We will handle this in Week 5; for Week 3, we focus on single-cycle dispatch only.

4.2 The Free List

The free list is a queue (FIFO) of physical register indices currently not mapped to any architectural register.

Initial state. If we have 32 architectural registers and 48 physical, then on reset:

- Rename map: arch reg R maps to physical R (the first 32 physical registers serve as the initial mapping).
- Free list: contains physical registers 32 through 47 (the remaining 16 are available for allocation).

Allocation. On dispatch, the renamer pops one physical register from the free list per write. If the free list is empty, the renamer **must stall the front-end** — no instruction can be dispatched without a fresh physical register for its destination.

Deallocation. On commit, the ROB returns the *old* physical register (the one being displaced by this commit) to the free list. On flush, all the new physical registers allocated to flushed instructions are returned, and the old mappings restored.

4.3 Why Free List Discipline Matters

The free list must guarantee one invariant: **no physical register is ever simultaneously in the free list and in the rename map**. If this is violated, two architectural registers could end up sharing storage — corrupting data silently.

Easy ways to break this invariant:

- Allocating the same physical register to two architectural destinations in the same cycle (forgetting to advance the free-list head).
- Returning a physical register to the free list on commit *without* first verifying it’s not still mapped (e.g., returning the old phys reg before the commit actually happens).
- On flush, double-counting: returning both the new phys reg *and* the old phys reg to the free list, when only the new one should be returned.

These are the classic Week 6 bugs. They show up as “the register file has two architectural registers pointing to the same physical slot,” and the symptom is data corruption hours of cycles later. Prevention is to enforce the invariant carefully at every free-list operation.

4.4 A Cycle-Level View

At dispatch in cycle T:

1. Read rename map for source operands of all dispatched instructions.
2. Read rename map for the current mapping of each destination (this is `old_phys_dest`).
3. Pop physical registers from free list for destinations.
4. Write new mappings into rename map (committed at end of cycle T).
5. Allocate ROB entries; store old phys dest and new phys dest in the ROB entries (in Week 6).

At commit in cycle T:

1. Look at ROB head. If valid and ready, commit.
2. Push old phys dest (from the ROB entry) onto the free list.
3. Clear ROB head entry. Advance head pointer.

The rename map itself is **not** touched at commit time. It was already updated at dispatch and reflects the speculative state. Commit just makes that state “permanent” by freeing the displaced physical register.

Concept Check 3

1. Suppose 32 architectural and 48 physical registers. After dispatching 16 instructions that each write a destination, how many free physical registers are left (assuming no commits yet)? What does the front-end do at the 17th instruction?
2. Why must the rename map be read-during-write capable (combinational read with same-cycle write visible) in a 2-wide design? Why isn't this needed for single-issue?
3. On a branch misprediction, we walk the ROB from tail to the mispredicted branch. For each entry, we restore the rename map and return the new phys dest to the free list. Why *from tail to branch* (i.e., backwards) and not from branch to tail (forwards)?

5 Comprehensive Theory 4: Worked Example — Renaming in Action [~25 min]

Let's trace what happens in the renamer for a small sequence. This shows how rename map, free list, and ROB interact in practice. Single-issue (one instruction per cycle) for clarity. We'll do 2-wide in Week 5.

5.1 Setup

- 32 architectural registers (we'll use R0–R5 only, for brevity).
- 8 physical registers (P0–P7). **This is deliberately tight** to demonstrate free-list pressure; real PRAVAH will use 48 physical.
- Initial rename map: $R0 \rightarrow P0, R1 \rightarrow P1, \dots, R5 \rightarrow P5$.
- Initial free list: [P6, P7].
- ROB has 8 entries (ROB[0] through ROB[7]). Head = Tail = 0.

Code:

```
I1: add  R3, R1, R2      # R3 = R1 + R2
I2: add  R4, R3, R1      # R4 = R3 + R1 (depends on I1)
I3: add  R3, R2, R5      # R3 = R2 + R5 (overwrites I1's R3 -- WAW)
```

5.2 Cycle 1: Dispatch I1

Sources: R1 (look up: P1), R2 (look up: P2).

Destination: R3. Pop free list: P6. Update rename map: $R3 \rightarrow P6$.

Allocate ROB[0]: arch dest = R3, new phys dest = P6, old phys dest = P3 (the prior mapping of R3).

Rename Map	Free List	ROB[0]
R3 P6	[P7]	R3, new=P6, old=P3, not ready

I1 is sent down the back-end to execute. Its RS stores: src1=P1, src2=P2, dest=P6.

5.3 Cycle 2: Dispatch I2

Sources: R3 (look up: P6 — which is I1's destination! Note: not P3. The map already reflects I1's dispatch). R1 (look up: P1).

Destination: R4. Pop free list: P7. Update rename map: $R4 \rightarrow P7$.

Allocate ROB[1]: arch dest = R4, new phys dest = P7, old phys dest = P4.

Rename Map (changed)	Free List	ROB[1]
R3 P6, R4	P7	R4, new=P7, old=P4, not ready
(others unchanged)	[empty!]	

I2's RS stores: src1=P6 (it depends on I1), src2=P1, dest=P7. Note that I2 will wait for PRF[6].ready to be set before executing.

Free list is now empty. If I3 arrives, what happens?

5.4 Cycle 3: Attempt to Dispatch I3 — Stall!

The front-end requests dispatch. The renamer checks the free list: empty. The renamer signals the front-end: **stall** = 1. I3 waits.

This is the canonical structural hazard of out-of-order machines: **no free physical registers**. The processor can't keep dispatching. We're stuck until something commits and returns a physical register to the free list.

5.5 Cycle 4: I1 Completes; Cycle 5: I1 Commits

Suppose I1 executes in cycle 3 and finishes; its result is written to PRF[6], PRF[6].ready is set, and ROB[0].ready = 1 at end of cycle 3.

In cycle 4, the ROB head (entry 0) is valid and ready. Commit:

- Push old phys dest P3 back to free list.
- Clear ROB[0]. Head advances to 1.

Free List	ROB
[P3]	head=1, tail=2

I3 can now dispatch in cycle 5.

5.6 Cycle 5: Dispatch I3 (finally)

Sources: R2 (look up: P2), R5 (look up: P5).

Destination: R3. Pop free list: **P3** (which is the only entry currently in the free list, since I1's commit just returned it).

Update rename map: R3 → P3.

Allocate ROB[2]: arch dest = R3, new phys dest = P3, old phys dest = P6 (the current mapping of R3 before I3's write).

Look closely at **old_phys_dest** = P6, which is I1's destination. Why? Because I1 has already committed, but its destination physical register P6 is now "live" as the storage for architectural R3 (until I3 commits and displaces it). The rename map at the moment of I3's dispatch said R3 → P6.

When I3 commits later, P6 will be pushed back to the free list, ready to be reused.

Note on the recycling. Notice that physical register P3 is being reused: it was originally architectural R3's home at reset, then displaced by I1, then freed at I1's commit, then re-allocated to I3's destination. This is completely normal — the free list is meant to recycle physical registers. The numbering coincidence (it happens that R3 starts in P3 and gets back to P3 later) is just because of the identity-map reset convention.

5.7 Key Takeaways

- **WAW dissolved.** I1 wrote R3, I3 also wrote R3. Different physical registers (P6 and P3). The rename map at any moment points to the architecturally-correct one. No stall needed at issue.
- **Free list pressure is real.** With only 2 spare physical registers, the pipeline stalled at I3 until I1 committed. Real designs use 16–32 spares; PRAVAH should use 16 (48 phys - 32 arch).
- **Old phys dest matters for recovery.** If I3 were a speculative branch we needed to undo, restoring $R3 \rightarrow P6$ (the old mapping at I3's dispatch) is exactly the rollback we need.

Concept Check 4

1. In the worked example, I2 reads R3 and gets P6 as its source. After I3 dispatches (which overwrites R3 to P3), does I2's stored source change? Why or why not?
2. Suppose I3 mispredicts (well, I3 is an add, not a branch, but imagine for the sake of argument that we need to flush I3 and beyond). Walk through exactly what gets restored and freed.
3. Why does the worked example use 8 physical registers for 6 architectural? Why is this configuration intentionally tight?

6 Verilog Design Philosophy — Before You Write Any Code [~15 min]

This is your first time writing Verilog for this project. Before opening your editor, internalize a few principles. Skipping this section will cost you days.

6.1 We Are Using Verilog-2001, Not SystemVerilog

Important toolchain note. The Quartus Prime Lite + ModelSim-Intel Starter Edition toolchain has limited and sometimes-inconsistent SystemVerilog support. To avoid file-extension traps and synthesis surprises, we will use **plain Verilog-2001** throughout this project. Concretely:

- All RTL files are `.v` (not `.sv`).
- Use `reg` and `wire`, not `logic`.
- Use `always @(posedge clk)` for sequential, `always @*` for combinational. Do not use `always_ff` or `always_comb`.
- No `enum`, no SystemVerilog packed structs, no `automatic` functions (use ordinary Verilog `function/endfunction`).
- `$clog2` is acceptable (Quartus supports it).

If you’ve used SystemVerilog before, this will feel slightly verbose. Trust the constraint — it eliminates an entire class of cross-tool inconsistency bugs.

6.2 Hardware Is Not Software

The mental shift, in one paragraph:

In software, statements execute one after another. In hardware, every statement happens at the same time, every clock cycle. A line like “`if (cond) x = y;`” isn’t a step the CPU performs; it’s a description of a wire that, given the right inputs, will hold a particular value. You are not writing instructions to be executed — you are describing a circuit that will exist.

Concretely:

- A 48-entry register file with 4 read ports is *four physical paths through the storage*, all operating in parallel every cycle. The four reads don’t “happen one after another.”
- A “free-list pop” is a wire that selects the head of the queue and another wire that increments the head pointer, both completing in one clock edge.
- A “for loop” in Verilog (used inside `always` blocks) is not a loop in time. It’s a description that, at synthesis time, unrolls into N copies of the same hardware. The N copies all run in parallel.

6.3 Synthesizable vs. Non-synthesizable Verilog

Synthesizable (lives in `rtl/`): code that can become real gates on an FPGA. The constraints:

- No `$display`, `$monitor`, `$readmemh`, `$write`, `$random`, etc.
- No `initial` blocks (Quartus tolerates these for memory initialization but they're not portable; use a synchronous reset instead).
- No delays (`#10`).
- Every `always @*` block must assign every signal in every code path (no inferred latches).
- Every `case` statement should have a `default` branch.

Testbench (lives in `tb/`): code that drives inputs to your synthesizable module and checks outputs. Anything goes — `$display`, `initial` blocks, delays, file reads. This code never goes to hardware.

Separation matters. If you mix testbench code into your RTL, Quartus will complain when you try to synthesize. Worse, the simulation behavior may differ from the synthesized behavior, leading to confusing bugs.

6.4 Naming Conventions for Your Sanity

You'll have lots of signals. Adopt these conventions from day 1:

- `_i` suffix for module inputs: `clk_i`, `reset_i`, `rs1_i`.
- `_o` suffix for module outputs: `rd_data_o`, `stall_o`.
- `_next` for combinational “next state” signals: `head_next`.
- No suffix for registered state: `head` (the current registered value).
- Use UPPERCASE for parameters: `parameter NUM_PHYS_REGS = 48`.

These conventions take 30 seconds to learn and save hours of debugging when you can't figure out what's a wire vs. a register.

6.5 Blocking vs. Non-blocking Assignment

In synchronous logic (inside `always @(posedge clk)`), always use **non-blocking** (`<=`):

```
1 always @(posedge clk_i) begin
2     head <= head_next;           // non-blocking, correct
3     tail <= tail_next;
4 end
```

In combinational logic (inside `always @*`), use **blocking** (`=`):

```
1 always @* begin
2     head_next = head;           // blocking, correct
3     if (commit_valid)
4         head_next = head + 1;
5 end
```

Mixing these up is the most common source of “my simulation works but my synthesis is broken” or “my simulation gives non-deterministic results.” If in doubt: `<=` in sequential, `=` in combinational.

6.6 Reset Strategy

Every flip-flop in your design must have a defined reset value. Otherwise, on startup, your design may be stuck in garbage state forever.

For PRAVAH, use synchronous reset (sampled on the clock edge):

```
1 always @(posedge clk_i) begin
2     if (reset_i) begin
3         head <= 0;
4         tail <= 0;
5     end else begin
6         head <= head_next;
7         tail <= tail_next;
8     end
9 end
```

Concept Check 5

1. Why is `$display` disallowed in synthesizable RTL? What would it “mean” on real hardware?
2. In an `always @(posedge clk)` block, if you write `head <= head + 1;` followed by `head <= 0;` in the same block, what value does `head` take after the cycle? (This is bad practice — never write code like this — but it tests your understanding of non-blocking assignment ordering.)
3. Why must every `case` statement have a `default` branch in synthesizable RTL? What happens otherwise?

7 Module 1: `register_file.v` — The Physical Register File [~30 min]

This is the storage for all in-flight values. It is the first real hardware module you will write.

7.1 Specification

- Storage: 48 physical registers, each 32 bits wide, each with a 1-bit ready flag.
- Read ports: 4 (to support 2-wide superscalar; each instruction needs 2 source reads).
- Write ports: 2 (to support 2-wide superscalar; each instruction writes one destination).
- Reads are combinational. Each read port outputs both the value and the ready bit for the requested register.
- Writes are clocked. A write sets the value *and* sets the ready bit to 1 (the value has just become defined).
- Allocation port (separate): clears the ready bit for a register when the rename unit allocates it. This says “a new in-flight computation has claimed this register; its value is not yet defined.”
- Bypass behavior: **if a write port and a read port target the same address in the same cycle, the read should return the new data and ready=1** (write-before-read forwarding). This avoids a 1-cycle latency for back-to-back operations.

Two key design decisions you should understand:

1. **Why ready bits live in the register file, not in the reservation stations.** In classical Tomasulo, an RS held the operand value once captured. In the modern PRF design, the PRF *is* the storage. The RS just holds an index. So the RS needs a way to know “is the operand at index P ready?” — which means the PRF must expose ready bits as part of every read.
2. **Why allocation clears the ready bit and writes set it.** A physical register’s ready bit means “the value here is the result of a completed computation.” When the rename unit allocates a register for a new destination, the value is *about to be produced but doesn’t exist yet*, so we clear the ready bit. When the FU writes the result, we set it.

7.2 Port List

```
1 module register_file #(
2     parameter NUM_PHYS_REGS = 48,
3     parameter DATA_WIDTH   = 32,
4     parameter ADDR_WIDTH    = 6      // log2(48) rounded up
5 )(
6     input  wire             clk_i,
7     input  wire             reset_i,
8
9     // Four read ports (each returns value + ready bit)
10    input  wire [ADDR_WIDTH-1:0] rd_addr1_i,
11    input  wire [ADDR_WIDTH-1:0] rd_addr2_i,
12    input  wire [ADDR_WIDTH-1:0] rd_addr3_i,
13    input  wire [ADDR_WIDTH-1:0] rd_addr4_i,
14    output wire [DATA_WIDTH-1:0] rd_data1_o,
15    output wire [DATA_WIDTH-1:0] rd_data2_o,
16    output wire [DATA_WIDTH-1:0] rd_data3_o,
17    output wire [DATA_WIDTH-1:0] rd_data4_o,
18    output wire             rd_ready1_o,
19    output wire             rd_ready2_o,
20    output wire             rd_ready3_o,
21    output wire             rd_ready4_o,
22
23    // Two write ports (each sets value AND sets ready=1)
24    input  wire             wr_en1_i,
25    input  wire [ADDR_WIDTH-1:0] wr_addr1_i,
26    input  wire [DATA_WIDTH-1:0] wr_data1_i,
27
28    input  wire             wr_en2_i,
29    input  wire [ADDR_WIDTH-1:0] wr_addr2_i,
30    input  wire [DATA_WIDTH-1:0] wr_data2_i,
31
32    // Allocate port (clears ready bit when a register is newly
33    allocated)
34    input  wire             alloc_en_i,
35    input  wire [ADDR_WIDTH-1:0] alloc_addr_i
36 );
```

7.3 Why Each Port Looks the Way It Does

Why `ADDR_WIDTH = 6`? $\log_2(48) = 5.58$, rounded up to 6 bits. 6 bits can address 64 entries; we use only 48, which is fine. Always size address buses with `$clog2(NUM_PHYS_REGS)` in real Verilog — I’ve hard-coded 6 here for clarity.

Why 4 reads + 2 writes? Each dispatched instruction reads 2 sources and writes 1 destination. With 2-wide superscalar, that's 4 reads and 2 writes per cycle.

Why expose ready bits on read ports? The reservation stations need to know if their operands are ready. Rather than have RSs separately poll a “ready bit vector,” we expose ready alongside data on every read.

Why a separate alloc port? Allocation (clearing the ready bit when a destination is renamed) is logically different from writing (setting the value when execution completes). They happen at different times and from different sources (renamer vs. FU). Keeping them as separate ports makes the data flow clearer.

Why no read-enable? Reads are free (no state change). We always drive read addresses and accept the read data + ready; if no instruction needs the data, the consumer simply ignores it.

7.4 Implementation

```
1  // Storage: value array and ready-bit array
2  reg [DATA_WIDTH-1:0] regs  [0:NUM_PHYS_REGS-1];
3  reg                      ready [0:NUM_PHYS_REGS-1];
4
5  integer i;
6
7  // ---- Synchronous state update ----
8  always @(posedge clk_i) begin
9      if (reset_i) begin
10         for (i = 0; i < NUM_PHYS_REGS; i = i + 1) begin
11             regs[i]  <= 32'b0;
12             ready[i] <= 1'b1;    // At reset, all PRF entries
                                   // are "ready" (hold valid zero)
13         end
14     end else begin
15         // 1. Writes set value AND ready bit
16         if (wr_en1_i) begin
17             regs[wr_addr1_i] <= wr_data1_i;
18             ready[wr_addr1_i] <= 1'b1;
19         end
20         if (wr_en2_i) begin
21             regs[wr_addr2_i] <= wr_data2_i;
22             ready[wr_addr2_i] <= 1'b1;
23         end
24
25         // 2. Allocation clears the ready bit
26         //     A write to the same address in the same cycle
                wins
```

```

27      //      (the allocation should not happen
28      //      simultaneously with a write
29      //      to the same address by design, but we make
30      //      write win for safety).
31      if (alloc_en_i &&
32          !(wr_en1_i && (wr_addr1_i == alloc_addr_i)) &&
33          !(wr_en2_i && (wr_addr2_i == alloc_addr_i)))
34          begin
35              ready[alloc_addr_i] <= 1'b0;
36          end
37      end
38  end
39  end
40
41  // ---- Combinational reads with write-before-read bypass
42  // ----
43  // For each read port, check if a write this cycle targets
44  // the same address.
45  // If so, return the new value (ready=1). Otherwise, return
46  // the stored value.
47  assign rd_data1_o = (wr_en1_i && (wr_addr1_i == rd_addr1_i))
48  ? wr_data1_i :
49      (wr_en2_i && (wr_addr2_i == rd_addr1_i))
50      ? wr_data2_i :
51      regs[rd_addr1_i];
52
53  assign rd_ready1_o = (wr_en1_i && (wr_addr1_i == rd_addr1_i))
54  ? 1'b1 :
55      (wr_en2_i && (wr_addr2_i == rd_addr1_i))
56      ? 1'b1 :
57      ready[rd_addr1_i];
58
59  assign rd_data2_o = (wr_en1_i && (wr_addr1_i == rd_addr2_i))
60  ? wr_data1_i :
61      (wr_en2_i && (wr_addr2_i == rd_addr2_i))
62      ? wr_data2_i :
63      regs[rd_addr2_i];
64
65  assign rd_ready2_o = (wr_en1_i && (wr_addr1_i == rd_addr2_i))
66  ? 1'b1 :
67      (wr_en2_i && (wr_addr2_i == rd_addr2_i))
68      ? 1'b1 :
69      ready[rd_addr2_i];
70
71  assign rd_data3_o = (wr_en1_i && (wr_addr1_i == rd_addr3_i))
72  ? wr_data1_i :

```

```

57         (wr_en2_i && (wr_addr2_i == rd_addr3_i))
58         ? wr_data2_i :
59         regs[rd_addr3_i];
60
61     assign rd_ready3_o = (wr_en1_i && (wr_addr1_i == rd_addr3_i))
62     ? 1'b1 :
63     (wr_en2_i && (wr_addr2_i == rd_addr3_i))
64     ? 1'b1 :
65     ready[rd_addr3_i];
66
67     assign rd_data4_o = (wr_en1_i && (wr_addr1_i == rd_addr4_i))
68     ? wr_data1_i :
69     (wr_en2_i && (wr_addr2_i == rd_addr4_i))
70     ? wr_data2_i :
71     regs[rd_addr4_i];
72
73     assign rd_ready4_o = (wr_en1_i && (wr_addr1_i == rd_addr4_i))
74     ? 1'b1 :
75     (wr_en2_i && (wr_addr2_i == rd_addr4_i))
76     ? 1'b1 :
77     ready[rd_addr4_i];
78
79 endmodule

```

7.5 What Could Go Wrong

Double-write to same address. What if `wr_en1 && wr_en2 && (wr_addr1 == wr_addr2)`?

This is a structural hazard at the write ports. In a correct PRAVAH design, the rename unit guarantees this never happens (different destinations get different physical registers). Your testbench should include an assertion to catch it if it does (see Section 8).

Write vs. alloc race. If a write and an allocate target the same address in the same cycle, the write wins (set value AND set ready) and the alloc has no effect. The conditional checks in the alloc branch handle this. This shouldn't happen by design (you don't allocate a register the FU is currently writing), but the priority is defined for safety.

Read-during-reset. During reset, the storage contents are being initialized to zero with `ready=1`. Your testbench should hold reset for several cycles before issuing reads to be safe.

The verbose bypass logic. The bypass code above is repetitive — four ports, two writes, eight nearly-identical `assign` statements. You could factor this into a Verilog `function`, but for first-time Verilog work, the explicit form is easier to debug. Pattern recognition over abstraction at this stage.

Concept Check 6

1. Why is the bypass logic (write-before-read) necessary? What would happen on back-to-back writes and reads to the same address without it?
2. Why does a write *set* the ready bit but allocation *clear* it? What invariant does this enforce?
3. How many physical wires does this 4R2W register file imply for the read data outputs alone? (Hint: $4 \text{ ports} \times \text{DATA_WIDTH}$ bits, plus 4 ready bits.)

8 Module 2: rename_unit.v — Rename Map + Free List [~35 min]

The rename unit translates architectural register references into physical register references at dispatch time. It owns the rename map and the free list.

8.1 Specification

- 32 architectural registers mapping to 48 physical registers.
- Single dispatch port (Week 3): takes one architectural destination and two architectural sources, returns the physical destination (allocated from free list), two physical sources (looked up in rename map), and the old physical destination (for ROB rollback).
- One commit/free port: when an instruction commits, returns its old physical destination to the free list.
- Stall output: signals 1 when dispatch needs a free physical register but the free list is empty.
- For Week 3, single-issue is sufficient. Week 5 expands to 2-wide.

8.2 Port List

```
1 module rename_unit #(
2     parameter NUM_ARCH_REGS = 32,
3     parameter NUM_PHYS_REGS = 48,
4     parameter ARCH_ADDR_W   = 5,                                //
5     parameter PHYS_ADDR_W   = 6,                                //
6     parameter FREE_LIST_SZ  = NUM_PHYS_REGS - NUM_ARCH_REGS // 16
7 ) (
8     input  wire          clk_i,
9     input  wire          reset_i,
10
11     // Dispatch port (rename one instruction per cycle)
12     input  wire          disp_valid_i,
13     input  wire [ARCH_ADDR_W-1:0] disp_rs1_arch_i,
14     input  wire [ARCH_ADDR_W-1:0] disp_rs2_arch_i,
15     input  wire [ARCH_ADDR_W-1:0] disp_rd_arch_i,
16     input  wire          disp_writes_rd_i,
17
18     output wire [PHYS_ADDR_W-1:0] disp_rs1_phys_o,
```

```

19     output wire [PHYS_ADDR_W-1:0]    disp_rs2_phys_o,
20     output wire [PHYS_ADDR_W-1:0]    disp_rd_phys_o,
21     output wire [PHYS_ADDR_W-1:0]    disp_rd_old_phys_o,
22     output wire                      stall_o,
23
24     // Commit port (return old phys dest to free list)
25     input  wire                      commit_valid_i,
26     input  wire [PHYS_ADDR_W-1:0]    commit_old_phys_i
27 );

```

8.3 Why Each Port Looks the Way It Does

Why `disp_writes_rd_i`? Some instructions (like `sw` or `beq`) don't write a destination. In that case, we don't pop the free list and don't update the rename map. Without this signal, you'd waste physical registers on non-writing instructions and exhaust the free list within a few hundred instructions.

Why `disp_rd_old_phys_o`? The ROB (Week 6) needs the old mapping of `rd` to roll back on misprediction. The renamer reads the old mapping (from the rename map, before overwriting it) and ships it to the ROB along with the new mapping.

Why `stall_o`? If the free list is empty and the instruction needs a destination, we can't proceed. The front-end must stall (hold the instruction in decode and don't advance).

Why the commit port is separate from dispatch? Commit and dispatch are independent events. Commit happens at the ROB head; dispatch happens at the front-end. They can both occur in the same cycle (in which case the free-list state must reflect both).

8.4 Internal Structure

```

1     // Rename map: 32 entries, each PHYS_ADDR_W bits
2     reg [PHYS_ADDR_W-1:0] rename_map [0:NUM_ARCH_REGS-1];
3
4     // Free list: circular FIFO of free physical registers
5     reg [PHYS_ADDR_W-1:0] free_list [0:FREE_LIST_SZ-1];
6
7     // Pointers: extra bit beyond log2(SIZE) to distinguish empty
8     vs full
9     reg [$clog2(FREE_LIST_SZ):0] fl_head;
10    reg [$clog2(FREE_LIST_SZ):0] fl_tail;
11    reg [$clog2(FREE_LIST_SZ):0] fl_count;

```

8.5 Implementation

```

1  // ---- Helper signals: do we actually dispatch / commit this
   // cycle? ----
2  wire do_dispatch = disp_valid_i & disp_writes_rd_i & ~stall_o
   ;
3  wire do_commit   = commit_valid_i;
4
5  // ---- Combinational outputs ----
6  assign disp_rs1_phys_o    = rename_map[disp_rs1_arch_i];
7  assign disp_rs2_phys_o    = rename_map[disp_rs2_arch_i];
8  assign disp_rd_old_phys_o = rename_map[disp_rd_arch_i];
9  assign disp_rd_phys_o     = free_list[fl_head[$clog2(
   FREE_LIST_SZ)-1:0]];
10
11 assign stall_o = disp_valid_i & disp_writes_rd_i & (fl_count
   == 0);
12
13 // ---- Synchronous state update ----
14 integer i;
15 always @(posedge clk_i) begin
16     if (reset_i) begin
17         // Identity map: arch reg R -> phys reg R
18         for (i = 0; i < NUM_ARCH_REGS; i = i + 1)
19             rename_map[i] <= i[PHYS_ADDR_W-1:0];
20         // Free list contains the upper physical registers
21         for (i = 0; i < FREE_LIST_SZ; i = i + 1)
22             free_list[i] <= NUM_ARCH_REGS[PHYS_ADDR_W-1:0] +
23                 i[PHYS_ADDR_W-1:0];
24         fl_head <= 0;
25         fl_tail <= 0;
26         fl_count <= FREE_LIST_SZ[$clog2(FREE_LIST_SZ):0];
27     end else begin
28         // Dispatch: pop free list, update rename map
29         if (do_dispatch) begin
30             rename_map[disp_rd_arch_i] <=
31                 free_list[fl_head[$clog2(FREE_LIST_SZ)-1:0]];
32             fl_head <= (fl_head + 1) % FREE_LIST_SZ;
33         end
34
35         // Commit: push to free list
36         if (do_commit) begin
37             free_list[fl_tail[$clog2(FREE_LIST_SZ)-1:0]] <=
38                 commit_old_phys_i;
39             fl_tail <= (fl_tail + 1) % FREE_LIST_SZ;
40         end

```

```

40
41      // Count update: single, clean expression
42      // -1 if we dispatched, +1 if we committed, 0 if
43      //   neither/both
44      fl_count <= fl_count
45                  - (do_dispatch ? 1 : 0)
46                  + (do_commit  ? 1 : 0);
47
48   end
49 endmodule

```

8.6 The Subtle Bits

Same-cycle dispatch and commit. The `do_dispatch/do_commit` helper wires let the `fl_count` update be a single, obviously-correct line. If both happen, count stays the same (we popped one and pushed one — and the head and tail pointers each advance by 1, so the queue remains coherent). If only one happens, count changes by 1.

Free list as circular buffer. We use a queue with head and tail pointers. Pop from head, push to tail. The `% FREE_LIST_SZ` makes it circular. The pointers are one bit wider than $\log_2(\text{SIZE})$ so we can distinguish “empty” (head=tail, count=0) from “full” (head=tail, count=SIZE) — though here we use `fl_count` directly for that, so the extra bit is just convention.

Reset values. The rename map is initialized to identity (arch $R \rightarrow$ phys R for $R \in [32]$). The free list contains the upper physical registers (32 through 47). `fl_count` starts at 16.

The stall_o condition. Only stall if (a) the front-end wants to dispatch and (b) the instruction needs a phys reg and (c) the free list is empty. Reads and non-writing instructions don’t stall the free list.

Why no do_dispatch dependence on commit_valid_i for the stall. In principle, if a commit returns a register this cycle, we could dispatch even though `fl_count == 0` at the start of the cycle. But this requires same-cycle bypass logic that complicates the design without much benefit. We accept a one-cycle bubble in the rare case of simultaneous empty-stall and commit.

8.7 What Could Go Wrong

Forgetting non-writing instructions. If `sw` doesn’t write a destination but you still pop the free list for it, you’ll exhaust your free list within a few hundred instructions and the pipeline will deadlock. The `disp_writes_rd_i` signal is non-negotiable.

Double-popping. If two dispatches happen in the same cycle (in the future 2-wide version), each must pop a *different* entry. We will address this in Week 5.

Out-of-order commits. Real ROBs may commit multiple instructions per cycle. If two commits happen, two pushes to the free list. The simple commit port shown here only supports one commit per cycle — enough for Week 3.

Concept Check 7

1. Walk through what `fl_count` should be after: reset, then 3 dispatches (each writing a destination), then 2 commits. Show the arithmetic.
2. If `disp_rd_arch_i = 5` and at the moment of dispatch `rename_map[5] = P#17`, what is the value of `disp_rd_old_phys_o`? What is the value of `rename_map[5]` *after* the clock edge?
3. Why does the free list use both `fl_head` and `fl_tail`, rather than just one pointer like a stack?

9 Testbenches — Verifying Your Modules [~25 min]

A module without a testbench is unverified, and an unverified module is broken until proven otherwise. Both modules need thorough testbenches before you push to GitHub.

9.1 What Makes a Good Testbench

1. **Self-checking.** Don't just print values — compare against expected values and print PASS/FAIL.
2. **Covers tricky cases.** Boundary conditions, simultaneous events, reset behavior.
3. **Easy to extend.** As you find bugs, add tests so they can't regress.
4. **Tells you where it failed.** “Test 3 failed: expected 42, got 17.”

9.2 Testbench Skeleton (Register File)

```
1  `timescale 1ns / 1ps
2
3  module tb_register_file();
4
5      // DUT signals
6      reg          clk, reset;
7      reg [5:0]    rd_addr1, rd_addr2, rd_addr3, rd_addr4;
8      wire [31:0]  rd_data1, rd_data2, rd_data3, rd_data4;
9      wire         rd_ready1, rd_ready2, rd_ready3, rd_ready4;
10     reg          wr_en1, wr_en2;
11     reg [5:0]    wr_addr1, wr_addr2;
12     reg [31:0]   wr_data1, wr_data2;
13     reg          alloc_en;
14     reg [5:0]    alloc_addr;
15
16     // Instantiate DUT
17     register_file dut (
18         .clk_i(clk), .reset_i(reset),
19         .rd_addr1_i(rd_addr1), .rd_addr2_i(rd_addr2),
20         .rd_addr3_i(rd_addr3), .rd_addr4_i(rd_addr4),
21         .rd_data1_o(rd_data1), .rd_data2_o(rd_data2),
22         .rd_data3_o(rd_data3), .rd_data4_o(rd_data4),
23         .rd_ready1_o(rd_ready1), .rd_ready2_o(rd_ready2),
24         .rd_ready3_o(rd_ready3), .rd_ready4_o(rd_ready4),
25         .wr_en1_i(wr_en1), .wr_addr1_i(wr_addr1), .wr_data1_i(
26             wr_data1),
27         .wr_en2_i(wr_en2), .wr_addr2_i(wr_addr2), .wr_data2_i(
28             wr_data2),
```

```

27     .alloc_en_i(alloc_en), .alloc_addr_i(alloc_addr)
28 );
29
30 // Clock
31 always #5 clk = ~clk;    // 100 MHz
32
33 // Self-checking helper
34 integer errors = 0;
35 task check_data;
36     input [31:0] actual;
37     input [31:0] expected;
38     input [255:0] label;
39     begin
40         if (actual === expected)
41             $display("PASS: %s (got %h)", label, actual);
42         else begin
43             $display("FAIL: %s (expected %h, got %h)", label,
44                 expected, actual);
45             errors = errors + 1;
46         end
47     end
48 endtask
49
50 initial begin
51     // Init
52     clk = 0; reset = 1;
53     wr_en1 = 0; wr_en2 = 0; alloc_en = 0;
54     rd_addr1 = 0; rd_addr2 = 0; rd_addr3 = 0; rd_addr4 = 0;
55     #20; reset = 0;
56
57     // Test 1: Read after reset -> 0
58     rd_addr1 = 6'd5;
59     #1;
60     check_data(rd_data1, 32'h0, "Read after reset");
61
62     // Test 2: Write then read
63     @(posedge clk);
64     wr_en1 <= 1; wr_addr1 <= 6'd5; wr_data1 <= 32'hDEAD_BEEF;
65     @(posedge clk);
66     wr_en1 <= 0;
67     #1;
68     rd_addr1 = 6'd5;
69     #1;
70     check_data(rd_data1, 32'hDEAD_BEEF, "Write then read");

```

```

71 // Test 3: Write-before-read bypass
72 @(posedge clk);
73 wr_en1 <= 1; wr_addr1 <= 6'd10; wr_data1 <= 32'hCAFE_BABE
    ;
74 rd_addr1 <= 6'd10;
75 #1;
76 check_data(rd_data1, 32'hCAFE_BABE, "Write-before-read
    bypass");
77
78 // Test 4: Simultaneous writes to different addresses
79 @(posedge clk);
80 wr_en1 <= 1; wr_addr1 <= 6'd20; wr_data1 <= 32'h1111_1111
    ;
81 wr_en2 <= 1; wr_addr2 <= 6'd21; wr_data2 <= 32'h2222_2222
    ;
82 @(posedge clk);
83 wr_en1 <= 0; wr_en2 <= 0;
84 rd_addr1 = 6'd20; rd_addr2 = 6'd21;
85 #1;
86 check_data(rd_data1, 32'h1111_1111, "Simultaneous write
    port 1");
87 check_data(rd_data2, 32'h2222_2222, "Simultaneous write
    port 2");
88
89 // Test 5: Four simultaneous reads
90 rd_addr1 = 6'd5; rd_addr2 = 6'd10;
91 rd_addr3 = 6'd20; rd_addr4 = 6'd21;
92 #1;
93 check_data(rd_data1, 32'hDEAD_BEEF, "4-way read port 1");
94 check_data(rd_data2, 32'hCAFE_BABE, "4-way read port 2");
95 check_data(rd_data3, 32'h1111_1111, "4-way read port 3");
96 check_data(rd_data4, 32'h2222_2222, "4-way read port 4");
97
98 // Test 6: Allocate clears ready bit
99 @(posedge clk);
100 alloc_en <= 1; alloc_addr <= 6'd5;
101 @(posedge clk);
102 alloc_en <= 0;
103 #1;
104 rd_addr1 = 6'd5;
105 #1;
106 if (rd_ready1 === 1'b0)
107     $display("PASS: Allocate clears ready bit");
108 else begin

```

```

109         $display("FAIL: Allocate did not clear ready (got %b)
110             ", rd_ready1);
111         errors = errors + 1;
112     end
113
114     // Test 7: Write after allocate sets ready bit back
115     @(posedge clk);
116     wr_en1 <= 1; wr_addr1 <= 6'd5; wr_data1 <= 32'h99;
117     @(posedge clk);
118     wr_en1 <= 0;
119     #1;
120     rd_addr1 = 6'd5;
121     #1;
122     if (rd_ready1 === 1'b1)
123         $display("PASS: Write sets ready bit");
124     else begin
125         $display("FAIL: Write did not set ready (got %b)",
126             rd_ready1);
127         errors = errors + 1;
128     end
129
130     if (errors == 0)
131         $display("ALL TESTS PASSED");
132     else
133         $display("FAILED %0d TESTS", errors);
134     $finish;
135 end
136 endmodule

```

9.3 Required Test Cases for the Register File

1. Reset clears storage (read any address after reset $\rightarrow$ 0, ready bit set).
2. Single write then read returns the written value.
3. Write-before-read bypass: same-cycle write and read to same address returns new value with ready=1.
4. Two simultaneous writes to different addresses both succeed.
5. Four simultaneous reads from four different addresses all succeed.
6. Allocate clears the ready bit; subsequent read returns ready=0.
7. Write after allocate sets the ready bit back to 1.

9.4 Required Test Cases for the Rename Unit

1. Reset state: `rename_map[i] = i` for `i` in `[0, 31]`; `free_list` has 16 entries.
2. Single dispatch writing R3: returns `disp_rd_phys = first free phys (P#32)`, `disp_rd_old_phys = P#3`.
3. Back-to-back dispatches writing the same arch register: each gets a different physical register. **This is the most important test — it directly verifies WAW dissolution.**
4. Dispatch followed by commit returning `old_phys = P#3`: free list grows by 1.
5. Non-writing instruction (`disp_writes_rd_i = 0`): rename map and free list unchanged.
6. Free list exhaustion: dispatch 16 writes without any commits, then attempt a 17th → `stall_o = 1`.
7. Simultaneous dispatch and commit: `fl_count` stays the same.

9.5 Running the Testbench in ModelSim

```
cd sim/  
vlib work  
vlog ../rtl/register_file.v ../tb/tb_register_file.v  
vsim -c -do "run -all; quit" tb_register_file
```

If you want a GUI with waveforms:

```
vsim tb_register_file  
add wave -recursive *  
run -all
```

Save a screenshot of the waveform showing your test cases passing. Push to the `sim/` folder.

Concept Check 8

1. Why does the testbench use `<=` (non-blocking) for some assignments and `=` (blocking) for others? When is each correct?
2. The testbench uses `#1` delays before checking read data. Why? What would happen if we checked immediately after assigning the address?
3. Write a single sentence describing the most important test case in the rename unit testbench. Why is it the most important?

10 Week 3 Deliverables — What to Submit

In Your Repo

- `rtl/register_file.v` — 4R2W parameterizable register file with per-register ready bits and write-before-read bypass.
- `rtl/rename_unit.v` — single-issue rename unit with rename map, free list, and commit port.
- `tb/tb_register_file.v` — self-checking testbench with at least the 7 required test cases.
- `tb/tb_rename_unit.v` — self-checking testbench with at least the 7 required test cases.
- `sim/waveforms/` — screenshots showing key tests passing.
- `learning_log.md` — updated with this week’s notes.

What Mentor Will Check

1. **Compiles in Quartus.** Open a Quartus project, add your two RTL files, hit “Analysis & Synthesis.” If it doesn’t compile, the milestone fails.
2. **ModelSim simulation passes all required tests** with PASS prints (no FAILs).
3. **Back-to-back same-arch-reg test passes for the renamer.** This is the gate for Week 4.
4. **Code is in the right folders.** RTL in `rtl/`, testbenches in `tb/`. No `$display` in RTL.
5. **Pure Verilog-2001.** No `logic`, `always_ff`, `always_comb`, or SystemVerilog packed structs.
6. **Learning log shows real thinking.** Not just “I wrote `register_file.v`.” What was confusing? What did you try that didn’t work? What clicked for you?

Common Bugs to Watch For

- **Latch inferred** on the rename map because not every signal is assigned in every branch. Use `always @*` and assign defaults at the top of every combinational block.
- **Off-by-one in free list count.** Test the `dispatch+commit-same-cycle` case carefully.

- **Reset doesn't actually reset the rename map.** On power-up, the map must be $R \rightarrow R$ identity, not garbage.
- **Bypass logic missing.** Without it, reading a register the same cycle you write it returns the old value.
- **Ready bit not cleared on allocate.** If allocation doesn't clear the ready bit, consumers see "ready" for a register whose value hasn't been written yet — the most dangerous silent bug in OoO design.

Time Estimate

- Theory (Sections 1–4): 2.5–3 hours.
- Verilog reading (Sections 5–8): 1–1.5 hours.
- Coding the register file + testbench: 1.5–2 hours (most of which is testbench).
- Coding the rename unit + testbench: 2.5–3 hours.
- Debugging: 1–2 hours (be honest with yourself — it always takes longer than you think).

A Appendix: Solutions to Concept Checks

Try every question on your own first. The Verilog Concept Checks are especially important to think through before reading the answer.

Concept Check 1 (From Tags to Physical Registers)

1. Because every in-flight instruction that writes a register needs its own private storage — a physical register — to hold the result until commit. If you have only 32 physical registers (matching the architectural count), you can only have one in-flight write per architectural register. With 48 physical, you have 16 “extras” that can be allocated to in-flight writes. The number of in-flight writes is capped at (physical - architectural), so having more physical registers directly increases instruction-level parallelism.
2. The number of in-flight writes is at most (NUM_PHYS_REGS - NUM_ARCH_REGS), because every architectural register always needs at least one physical home (the current “committed” value). The remaining physical registers form the pool of available destinations.
3. P#5 is **not** freed immediately. It still holds the value that “architectural \$s0” had *before* this instruction. That old value might still be needed by other in-flight readers (instructions that read \$s0 before this rename happened, who captured tag P#5). P#5 is freed later, when the new instruction *commits* — at which point the rename map’s new mapping becomes permanent and P#5’s value is no longer architecturally meaningful.

Concept Check 2 (The Reorder Buffer)

1. By Little’s Law: throughput = in-flight / latency, so max IPC = 16 / 3 $\approx$ 5.3. (Real designs achieve much less due to RS stalls, branch mispredicts, etc.; the ROB size sets an upper bound.)
2. Because the architectural state changes only at commit. If we commit in order, the architectural state at any moment reflects exactly the prefix of the program that has fully committed. When an exception is raised, we can flush everything not yet committed, leaving state at a precise instruction boundary.
3. Two updates: (1) The new mapping in the rename map (arch dest = \$s0, points to new phys = P#17) becomes permanent — meaning no rollback could undo it. (2) The old physical register P#3 is pushed onto the free list, available for reallocation.

Concept Check 3 (Rename Map and Free List)

1. Starting with 16 spares ($48 - 32 = 16$). After 16 dispatches that each write, the free list is empty. The 17th would stall.
2. In single-issue, only one instruction dispatches per cycle, so its updates to the rename map are visible to the *next* cycle's dispatch without any bypass. In 2-wide, two instructions dispatch in the same cycle, and the second may depend on the first's update — which hasn't been clocked into the rename map yet. We need internal bypass logic so instruction 2 sees instruction 1's freshly-written mapping.
3. Because the order in which we apply rename-map restorations matters when multiple flushed instructions write to the same architectural register. Going tail-to-branch (newest-to-oldest), each rollback restores to the state *before* that instruction's dispatch. The chain of `old_phys_dest` values forms a correct walk-back through the rename history. Going branch-to-tail would restore overwritten mappings in the wrong order, ending up with the wrong final state.

Concept Check 4 (Worked Example)

1. I2's stored source does **not** change. At dispatch, I2 captured "`src1 = P6`." That field is fixed in I2's reservation station. Later changes to the rename map (e.g., R3 now points to P3 after I3's dispatch) do not retroactively modify I2. This is the renaming guarantee at work — I2 reads from PRF[6] when it eventually fires, getting I1's result.
2. On flushing I3: restore `rename_map[R3] → P6` (the value of `old_phys_dest` stored in I3's ROB entry). Push P3 (I3's `new_phys_dest`) back to the free list.
3. Because the worked example is intentionally tight on free list pressure to demonstrate the stall mechanism. Real designs have 16+ spare physical registers to avoid frequent stalls; PRAVAH uses $48 - 32 = 16$.

Concept Check 5 (Verilog Design Philosophy)

1. Because `$display` writes to the simulator's console. On real hardware, there is no console. The compiler cannot synthesize "write a string to the console" into gates — it's not a hardware operation. RTL must describe physical behavior; `$display` is purely a software simulation construct.
2. In Verilog's non-blocking assignment, the last assignment to a given signal in a block wins (the assignments are evaluated in order, but only the final scheduled update takes effect at the end of the time step). So `head` would become 0. **Never write code like this** — the result is technically deterministic in Verilog but extremely confusing, and may behave differently across simulators. Always assign each signal exactly once per block.

3. Without a default branch, a `case` that doesn't cover all input combinations becomes a latch — the compiler infers “hold the previous value when no case matches.” Latches in synchronous designs are extremely dangerous (no clock control over them) and Quartus will warn about them. Always include `default`.

Concept Check 6 (Register File)

1. Without bypass, a write at clock edge T and a read of the same address at clock edge $T+1$ would see the new value (good). But a read *within* the same cycle as the write sees the *old* value, because the write hasn't latched yet. This 1-cycle latency would force every dependent instruction to wait an extra cycle — catastrophic for IPC. Bypass eliminates the gap.
2. Write sets `ready=1` because the FU has just produced a valid value. Allocate clears `ready=0` because the rename unit has just claimed this register for a future in-flight instruction whose value hasn't been computed. The invariant: *ready=1 iff the value in the register is the result of a completed computation*. Consumers (RSs) only fire when both source operands have `ready=1`.
3. $4 \text{ ports} \times 32 \text{ bits} = 128 \text{ data wires} + 4 \text{ ready bits} = 132 \text{ read-output wires}$. Plus $4 \times 6 = 24 \text{ input address wires}$. The read interface alone is over 150 wires.

Concept Check 7 (Rename Unit)

1. After reset, `fl_count = 16`. After 3 dispatches: 13. After 2 commits: 15. Net: $16 - 3 + 2 = 15$.
2. `disp_rd_old_phys_o = P#17` (the current mapping). After the clock edge, `rename_map[5]` = the value of `free_list[fl_head]` at the moment of dispatch (whatever the next free physical register was).
3. Because we want FIFO behavior, not LIFO. A pushed-back physical register should not be immediately re-used in the next dispatch — it should wait its turn at the back of the queue. This avoids pathological cases where a just-freed register might be reused while some long-tail snooping logic still cares about its previous tag.

Concept Check 8 (Testbenches)

1. `<=` (non-blocking) is for driving DUT inputs from the testbench in a synchronous fashion (mimicking real hardware behavior). `=` (blocking) is for setting up combinational addresses or doing checks where you want immediate visibility. Inside `always @(posedge clk)`, always non-blocking. Inside `initial` blocks, either can be used, but stick to non-blocking for inputs that mimic clocked behavior.

2. Because reads are combinational — the read data settles *after* the address propagates through the logic. Checking immediately would race the assignment. The #1 gives the simulator one timestep to propagate.
3. “Back-to-back dispatches writing the same arch register must get different physical registers.” Because this single test directly verifies the WAW-dissolution mechanism that is the entire point of register renaming. If this fails, your renamer is broken, full stop.